# 2. Causal Models and Causal Graphs

A consolidated, mathematical lecture companion

English notes based on the lecture slides *Causal Model & Causal Graph* (Prof. Sanghack Lee, GSDS SNU)

The central question is what object can play the role of an "oracle" for causal questions, given that the observational distribution alone cannot determine the right decision, and what that object implies about the data we do observe.

These notes consolidate repeated slides, expand derivations (factorizations, conditional independences, d-separation), and add worked answers to the in-class exercises. Physical PDF page numbers of the source are listed in Section 8.

## Contents

**1 Reviews: notation, probability, algebra** **2**

1.1 Notation . . . 2

1.2 Probability . . . 2

1.3 Sums of products . . . 2

**2 Why data alone is not enough: a prima facie example** **3**

2.1 The story . . . 3

2.2 Current protocol . . . 3

2.3 Hypothetical protocols . . . 3

2.4 What can a learning system see? . . . 3

2.5 From statistical to causal analysis . . . 4

2.6 Modeling reality in the example . . . 4

**3 Structural causal models** **4**

3.1 Property 1: an SCM induces the observational distribution . . . 5

3.2 Property 2: an SCM induces a causal diagram . . . 5

3.3 Graph terminology . . . 6

3.4 Property 3: an SCM induces Pearl's Causal Hierarchy . . . 6

**4 The first layer: what the diagram encodes about $P(\mathbf{v})$** **6**

**5 Conditional independence and the Markov condition** **7**

5.1 Conditional independence . . . 7

5.2 Local Markov condition . . . 8

5.3 Markovian models and the Bayesian factorization . . . 8

**6 Reading independences from the graph: d-separation** **8**

6.1 Chains . . . 8

6.2 Common causes (forks) . . . 9

6.3 Common effects (colliders) . . . 9

6.4 Triplet summary . . . 9

6.5 d-separation . . . 9

**7 Implementation: graph algorithms in Python** **10**

7.1 Topological order . . . 10

7.2 d-separation via reachability (Markovian graphs) . . . 11

**8 Source coverage** **11**

# 1 Reviews: notation, probability, algebra

**Purpose.** Fix notation and the elementary manipulations used throughout the course.

## 1.1 Notation

- A random variable is denoted by an uppercase letter, e.g. $Z$, with state space $\mathfrak{X}_Z$ (rarely written explicitly). A lowercase letter denotes a value, $z \in \mathfrak{X}_Z$.
- Bold letters denote sets of variables or values, e.g. $\mathbf{V}$ and $\mathbf{v}$. Calligraphic letters denote complex objects such as graphs $\mathcal{G}$ and models $\mathcal{M}$.
- $\forall$ (for all), $\exists$ (exists); $\cup, \cap$; $\subseteq, \subset$ (note that $\subset$ is ambiguous between proper and non-proper subset); $\vee$ (or), $\wedge$ (and), $\oplus$ (xor); $\sum$, $\prod$; $\langle \cdots \rangle$ tuples, $\{\cdots\}$ sets, $\mid$ "given".
- "iff" = if and only if. $p \Rightarrow q$: if $p$ then $q$; $p$ is *sufficient* for $q$. Equivalently $\neg q \Rightarrow \neg p$: $q$ is *necessary* for $p$.

## 1.2 Probability

Consider $P(A, B, C, D, E, F)$; write $P(A = a, B = b)$ as $P(a, b)$. Variables are assumed discrete without loss of generality.

**Marginalization.**

$$\sum_{c,e} P(a, b, c, d, e, f) = \sum_{(c,e) \in \mathfrak{X}_C \times \mathfrak{X}_E} P(a, b, c, d, e, f) = P(a, b, d, f).$$

**Conditioning.**

$$P(a, b, c \mid d, e) = \frac{P(a, b, c, d, e)}{P(d, e)} = \frac{\sum_f P(a, b, c, d, e, f)}{\sum_{a,b,c} \sum_f P(a, b, c, d, e, f)}.$$

**Chain rule.** Applying the definition of conditional probability recursively, in *any* order:

$$P(a, b, d, e) = P(a \mid b, d, e) P(b \mid d, e) P(d \mid e) P(e) = P(b \mid a, d, e) P(d \mid a, e) P(e \mid a) P(a).$$

**Axioms.** $0 \leq P(\cdot) \leq 1$ and $\sum_{a,b,c} P(a, b, c) = 1$. Often strictly positive distributions are assumed so that every conditional is defined.

## 1.3 Sums of products

Factors that do not depend on a summation index can be moved outside that sum:

$$\sum_{a,c} f(a, b) g(b, c) h(c, d) = \sum_c h(c, d) g(b, c) \Big( \sum_a f(a, b) \Big) = \sum_a f(a, b) \Big( \sum_c g(b, c) h(c, d) \Big).$$

If $f(a, b) = P(a \mid b)$, then $\sum_a P(a \mid b) = 1$ and

$$\sum_a P(a \mid b) \underbrace{\sum_c g(b, c) h(c, d)}_{\text{irrelevant to } a} = \underbrace{\Big( \sum_a P(a \mid b) \Big)}_{=1} \sum_c g(b, c) h(c, d) = \sum_c g(b, c) h(c, d).$$

This "sum out a conditional of a leaf" step is used constantly in identification proofs.

# 2 Why data alone is not enough: a prima facie example

**Purpose.** Construct a situation in which every learning algorithm, trained on observational data, gives the wrong policy recommendation.

## 2.1 The story

A city suffers from a contagious disease, and a drug is *believed* to help. Unknown to physicians:

- People with good living conditions (rich) always survive.
- Some people carry a gene that naturally fights the disease; they need no treatment. If treated, however, they develop an allergic reaction that is fatal under poor living conditions.

Being rich and having the gene are independent, $P(\text{rich}) = P(\text{poor}) = \frac{1}{2}$, and half the poor have the gene. The population $\Pi$ splits into three groups:

$$\Pi = \text{rich} \ \cup \ \text{poor}_1 \ \cup \ \text{poor}_2,$$

$$\begin{cases} \text{rich}: & \text{alive anyway}, \\ \text{poor}_1 \ (\text{no gene}): & \text{die anyway}, \\ \text{poor}_2 \ (\text{gene}): & \text{die iff they take the drug}, \end{cases}$$

with $P(\text{poor}_1) = P(\text{poor}_2) = \frac{1}{4}$. (*Prima facie* = at first sight.)

## 2.2 Current protocol

Because the drug is expensive, physicians prescribe it to every rich person and only to them. Observed outcomes: all treated (rich) survive; among the untreated (poor) $\text{poor}_2$ survive and $\text{poor}_1$ die. Hence

$$P(\text{alive} \mid \text{drug}) = 1, \qquad P(\text{alive} \mid \neg\text{drug}) = 0.5.$$

*Should the city give the drug to everyone for free?*

## 2.3 Hypothetical protocols

| Group (mass) | Current (drug iff rich) | Drug for all | Drug for none |
|---|---|---|---|
| rich (1/2) | alive | alive | alive |
| $\text{poor}_1$ (1/4) | dead | dead | dead |
| $\text{poor}_2$ (1/4) | alive | dead | alive |
| $P(\text{alive})$ | 3/4 | 1/2 | 3/4 |

Giving the drug to everyone *reduces* survival from 3/4 to 1/2; the drug never helps anyone. The apparent benefit is produced entirely by the fact that only the rich, who survive anyway, are treated.

## 2.4 What can a learning system see?

A data set has rows (patient, $R$, $D$, $A$), e.g. (1, yes, yes, yes), (2, no, no, yes), (3, no, no, no), (4, yes, yes, yes),... The best any algorithm (decision tree, SVM, DNN) can do is to learn the joint distribution

| $R$ | $D$ | $A$ | $P(r,d,a)$ |
|---|---|---|---|
| 0 | 0 | 0 | 1/4 |
| 0 | 0 | 1 | 1/4 |
| 1 | 1 | 1 | 1/2 |
| others | | | 0 |

What remains unseen are the exogenous factors: the probability of having the allergic gene, $P$(gene); the other factors that affect wealth; and how the genetic factor interacts with wealth, the drug and the reaction.

**What is missing?** Consider a second world in which the drug is simply inert, $A \leftarrow R \vee U_G$. Since $D = R$ in the data, this world produces exactly the same table, but there "drug for all" gives survival 3/4. Two realities with different answers generate identical data, so no algorithm using only $P(r,d,a)$ can be correct in both. The data must be supplemented with a model of how the data are generated.

### 2.5 From statistical to causal analysis

- **Traditional statistics/ML:** data $\rightarrow$ joint distribution $P \rightarrow$ query $Q(P)$, an aspect of $P$, e.g. will customers who bought $A$ also buy $B$, $Q = P(B \mid A)$.
- **Causal analysis:** data $\rightarrow P \xrightarrow{\text{change}} P' \rightarrow Q(P')$, e.g. $P'$(sales) if we double the price, $P'$(cancer) if smoking is banned. How does the factual $P$ change into the hypothetical $P'$? $P$ is tied to data; $P'$ is never observed. A new formalism must represent both.
- **SCM paradigm:** data $\leftarrow P \leftarrow \mathcal{M}$ (data-generating model) $\rightarrow Q(\mathcal{M})$. $\mathcal{M}$ is an invariant strategy (mechanism, recipe, law, protocol) by which Nature assigns values to the variables. *$P$ is a model of data; $\mathcal{M}$ is a model of reality.*

In the big picture of Lecture 1, knowledge of the real world enters the causal model, and causal inference draws new knowledge about not-yet-realized alternative realities. (The slide warns: take this picture with a grain of salt.)

### 2.6 Modeling reality in the example

Observed (endogenous) variables $\mathbf{V}$: $R$ (1 = rich), $D$ (1 = took the drug), $A$ (1 = alive). Unobserved (exogenous) variables $\mathbf{U}$: $U_G$ (1 = has the genetic factor), $U_R$ (other factors affecting wealth). Mechanisms:

$$R \leftarrow U_R, \qquad D \leftarrow R, \qquad A \leftarrow R \vee (U_G \wedge \neg D),$$

with $P(U_G = 1) = P(U_R = 1) = \frac{1}{2}$ and $P(\mathbf{U}) = P(U_G)P(U_R)$. This is a fully specified model of reality: it implies both $P$ and every $P'$. It is our new oracle, the *structural causal model*.

## 3 Structural causal models

**Purpose.** Define the object that represents reality and derive its three basic properties.

**Definition 3.1** (Structural causal model; Pearl 2000)**.** An SCM $\mathcal{M}$ is a 4-tuple $\langle \mathbf{V}, \mathbf{U}, \mathcal{F}, P(\mathbf{U}) \rangle$ where

- $\mathbf{V} = \{V_1, \ldots, V_n\}$ are endogenous variables;

- $\mathbf{U}$ is a set of exogenous variables;
- $\mathcal{F} = \{f_1, \ldots, f_n\}$ are functions determining $\mathbf{V}$, $v_i \leftarrow f_i(\mathbf{pa}_i, \mathbf{u}_i)$, with $Pa_i \subseteq \mathbf{V} \setminus \{V_i\}$ and $\mathbf{U}_i \subseteq \mathbf{U}$;
- $P(\mathbf{U})$ is a distribution over $\mathbf{U}$.

The arrow "$\leftarrow$" stresses that $f_i$ is an assignment (a mechanism), not a symmetric equation. Axiomatic characterizations of the resulting counterfactual logic are given by Galles and Pearl (1998) and Halpern (1998). Abstractly: the exogenous $\mathbf{U}$ carry all randomness; $\mathcal{F}$ deterministically transforms a unit $\mathbf{u}$ into its observable profile.

### 3.1 Property 1: an SCM induces the observational distribution

$\mathcal{F}$ is a map $\mathbf{u} \mapsto (v_1, \ldots, v_n)$. When $\mathbf{U}$ is random, so is $\mathbf{V}$. The induced $P(\mathbf{v})$ is Layer 1 of the PCH, the observational (passive) distribution. Each event, person or observation corresponds to an instantiation $\mathbf{U} = \mathbf{u}$.

**Definition 3.2** (Induced distribution)**.** $\mathcal{M}$ defines $P^{\mathcal{M}}(\mathbf{V})$ by, for each $\mathbf{Y} \subseteq \mathbf{V}$,

$$P^{\mathcal{M}}(\mathbf{y}) = \sum_{\{\mathbf{u} \,:\, \mathbf{Y}(\mathbf{u}) = \mathbf{y}\}} P(\mathbf{u}),$$

where $\mathbf{Y}(\mathbf{u})$, the *potential response*, is the value of $\mathbf{Y}$ obtained by solving $\mathcal{F}$ at $\mathbf{u}$.

**Example 3.3.** In the drug example each citizen falls into one of four groups:

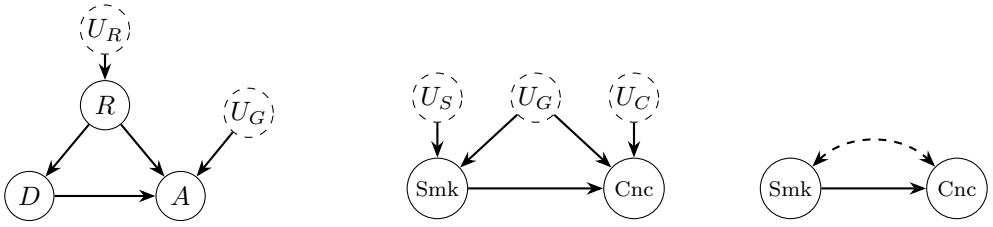
| $P(\mathbf{u})$ | $(U_R, U_G)$ | $\xrightarrow{\mathcal{F}} (R, D, A)$ | $P(\mathbf{v})$ |
|---|---|---|---|
| 1/4 | (1,1) | (1,1,1) | 1/2 |
| 1/4 | (1,0) | (1,1,1) | |
| 1/4 | (0,1) | (0,0,1) | 1/4 |
| 1/4 | (0,0) | (0,0,0) | 1/4 |

### 3.2 Property 2: an SCM induces a causal diagram

**Definition 3.4** (Causal diagram; Bareinboim et al. 2020, Def. 13)**.** Given $\mathcal{M} = \langle \mathbf{V}, \mathbf{U}, \mathcal{F}, P(\mathbf{U}) \rangle$, the causal diagram $\mathcal{G}$ is constructed by

1. adding a vertex for every $V_i \in \mathbf{V}$;
2. adding a directed edge $V_j \to V_i$ whenever $V_j$ is an argument of $f_i$;
3. adding a bidirected edge $V_j \leftrightarrow V_i$ whenever $\mathbf{U}_i$ and $\mathbf{U}_j$ are correlated or $f_i, f_j$ share some $U \in \mathbf{U}$ as an argument.

More precisely, for every set $\mathbf{W}$ of vertices with no bidirected edges among them, $\{\mathbf{U}_W\}_{W \in \mathbf{W}}$ are jointly independent. If the directed part is acyclic (assumed throughout) the diagram is a DAG with possible bidirected edges (an ADMG). For instance, $V_i \leftarrow f_i(A, B, U)$ and $V_j \leftarrow f_j(C, U)$ give $A \to V_i \leftarrow B$, $C \to V_j$, and $V_i \leftrightarrow V_j$ because of the shared $U$.




drug example explicit **U** **U** implicit

**Example 3.5** (Smoking and cancer)**.** $\mathbf{V} = \{\text{Smoking}, \text{Cancer}\}$, $\mathbf{U} = \{U_S, U_C, U_G\}$, Smoking $\leftarrow f_S(U_S, U_G)$, Cancer $\leftarrow f_C(\text{Smoking}, U_C, U_G)$. The shared genetic factor $U_G$ yields Smoking $\leftrightarrow$ Cancer.

**Remark 1.** The mapping goes one way only: a graph is compatible with infinitely many SCMs sharing the same function signatures and exogenous structure.
**Remark 2.** This is central: in practice researchers often know the *scope* of the mechanisms (which variables enter which function) but not their form. Causal inference asks what can be concluded from the graph plus data alone.
**Convention.** Unobserved variables are left implicit; a shared or correlated $U$ is drawn as a dashed bidirected edge.

### 3.3 Graph terminology

A graph $\mathcal{G} = \langle \mathbf{V}, \mathbf{E} \rangle$ consists of vertices (nodes) and edges. For $V \in \mathbf{V}$:

- parents $Pa(V) = \{W : W \to V \in \mathcal{G}\}$; children $Ch(V) = \{W : V \to W \in \mathcal{G}\}$;
- ancestors $An(V) = \{W : W \to \cdots \to V \in \mathcal{G}\}$; descendants $De(V) = \{W : V \to \cdots \to W \in \mathcal{G}\}$; these include $V$ itself (zero-length paths);
- siblings (spouses through confounding) $\{W : W \leftrightarrow V \in \mathcal{G}\}$;
- a *path* is a sequence of edges joining a sequence of distinct vertices; a *directed path* from $W$ to $V$ has all arrowheads pointing toward $V$.

### 3.4 Property 3: an SCM induces Pearl's Causal Hierarchy

An SCM $\mathcal{M}$ answers queries of all three layers: association $P(y \mid x)$, intervention $P(y \mid \text{do}(x), c)$ and counterfactual $P(y_x \mid x', y')$. This lecture concentrates on Layer 1; Layers 2 and 3 are formalized in Lecture 3.

## 4 The first layer: what the diagram encodes about $P(\mathbf{v})$

**Purpose.** Derive the factorization of the observational distribution implied by an SCM and its diagram.

$\mathcal{M}$ induces

$$P(\mathbf{v}) = \sum_{\mathbf{u}} P(\mathbf{V} = \mathbf{v}, \mathbf{U} = \mathbf{u}) = \sum_{\mathbf{u}} P\big(\forall_{V_i \in \mathbf{V}}\, f_i(\mathbf{pa}_i, \mathbf{u}_i) = v_i,\ \mathbf{U} = \mathbf{u}\big),$$

where $\mathbf{pa}_i, \mathbf{u}_i$ are the observed and unobserved arguments of $f_i$, i.e. the parents of $V_i$ in $\mathcal{G}$. In the

examples,

$$P(r,d,a) = \sum_{u_R,u_G} P(\underbrace{r,d,a}_{\text{endogenous}}, \underbrace{u_R,u_G}_{\text{exogenous}}),$$
$$P(s,c) = \sum_{u_S,u_G,u_C} P(s,c,u_S,u_G,u_C).$$

This is the observational (passive, non-experimental) distribution.

**Topological order.** Because $\mathcal{G}$ is acyclic, there is an order $\prec$ on $\mathbf{V}$ in which every variable comes after its parents, $Pa_i \prec V_i$; e.g. $R \prec D \prec A$ and $S \prec C$.

**Factorization.** Apply the chain rule along a topological order:

$$P(\mathbf{v}) = \sum_{\mathbf{u}} P(\mathbf{u}) \prod_{V_i \in \mathbf{V}} P(v_i \mid v_1, \ldots, v_{i-1}, \mathbf{u}).$$

Each $V_i$ is fully determined by its observed and unobserved parents, and $Pa_i \cup \mathbf{U}_i \subseteq \{V_1, \ldots, V_{i-1}\} \cup \mathbf{U}$, so

$$P(v_i \mid v_1, \ldots, v_{i-1}, \mathbf{u}) = P\big(f_i(\mathbf{pa}_i, \mathbf{u}_i) = v_i \mid v_1, \ldots, v_{i-1}, \mathbf{u}\big) = P(v_i \mid \mathbf{pa}_i, \mathbf{u}_i).$$

Therefore

$$\boxed{P(\mathbf{v}) = \sum_{\mathbf{u}} P(\mathbf{u}) \prod_{V_i \in \mathbf{V}} P(v_i \mid \mathbf{pa}_i, \mathbf{u}_i).}$$

Examples:

$$P(r,d,a) = \sum_{u_R,u_G} P(u_R,u_G)P(r \mid u_R)P(d \mid r)P(a \mid r,d,u_G),$$
$$P(s,c) = \sum_{u_S,u_G,u_C} P(u_S,u_G,u_C)P(s \mid u_G,u_S)P(c \mid s,u_G,u_C).$$

Here $P(v_i \mid \mathbf{pa}_i, \mathbf{u}_i) \in \{0,1\}$ because the mechanisms are deterministic.

# 5 Conditional independence and the Markov condition

**Purpose.** Translate the structure of $\mathcal{G}$ into testable constraints on $P(\mathbf{v})$.

## 5.1 Conditional independence

$X$ and $Y$ are independent, $X \perp\!\!\!\perp Y$, if learning $X = x$ does not change the belief in $Y = y$:

$$X \perp\!\!\!\perp Y \iff P(y \mid x) = P(y) \;\; \forall x, y \iff P(x,y) = P(x)P(y).$$

$X$ and $Y$ are conditionally independent given $Z$ if, once $Z = z$ is known, $X$ is uninformative about $Y$:

$$(X \perp\!\!\!\perp Y \mid Z) \iff P(x \mid z, y) = P(x \mid z) \quad \forall x, y, z \text{ with } P(y,z) > 0.$$

These are constraints on $P(\mathbf{V})$. The guiding intuition: *lack of functional dependence leads to probabilistic independence.*

### 5.2 Local Markov condition

In any topological order, $P(v_i \mid v_1, \ldots, v_{i-1}, \mathbf{u}) = P(v_i \mid \mathbf{pa}_i, \mathbf{u}_i)$. What is the largest set that can precede $V_i$ in some topological order? Any $W$ that is not reachable by a directed path $V_i \to \cdots \to W$, i.e. all *non-descendants* $Nd_i$ of $V_i$. Therefore

$$P(v_i \mid \mathbf{pa}_i, \mathbf{u}_i) = P(v_i \mid \mathbf{nd}_i, \mathbf{u}),$$

which is the statement

$$V_i \perp\!\!\!\perp (Nd_i \setminus Pa_i),\ (\mathbf{U} \setminus \mathbf{U}_i) \ \mid\ Pa_i, \mathbf{U}_i.$$

### 5.3 Markovian models and the Bayesian factorization

If $U_1, \ldots, U_n$ are jointly independent (no bidirected edges), the model is *Markovian*. Then

$$\begin{aligned}
P(\mathbf{v}) &= \sum_{\mathbf{u}} P(\mathbf{u}) \prod_i P(v_i \mid \mathbf{pa}_i, u_i) \\
&= \sum_{\mathbf{u}} \prod_i P(v_i \mid \mathbf{pa}_i, u_i) P(u_i) && (U_i \text{ independent}) \\
&= \sum_{\mathbf{u}} \prod_i P(v_i \mid \mathbf{pa}_i, u_i) P(u_i \mid \mathbf{pa}_i) && (U_i \perp\!\!\!\perp Pa_i) \\
&= \sum_{\mathbf{u}} \prod_i P(v_i, u_i \mid \mathbf{pa}_i) \\
&= \prod_i \sum_{u_i} P(v_i, u_i \mid \mathbf{pa}_i) = \prod_{V_i \in \mathbf{V}} P(v_i \mid \mathbf{pa}_i).
\end{aligned}$$

The penultimate step uses that the $i$-th factor depends only on $u_i$, so the sum over $\mathbf{u}$ distributes over the product. $U_i \perp\!\!\!\perp Pa_i$ holds because $Pa_i$ are functions of exogenous variables of non-descendants, which are independent of $U_i$ in a Markovian model.

**Summary.** Markovian SCM $\Rightarrow$ $P(\mathbf{v}) = \prod_i P(v_i \mid \mathbf{pa}_i)$ (Bayesian factorization) $\Leftrightarrow$ local Markov condition $V_i \perp\!\!\!\perp Nd_i \setminus Pa_i \mid Pa_i$.

## 6 Reading independences from the graph: d-separation

**Purpose.** Connect the unobserved mechanisms with observable independences through three elementary structures.

Each example below uses binary variables and explicit mechanisms.

### 6.1 Chains

$X \to Z \to Y$: good study habits $X \leftarrow U_X$, a high Suneung score $Z \leftarrow X \vee \neg U_Z$ ($U_Z$: difficulty), college admission $Y \leftarrow Z \wedge U_Y$ ($U_Y$: favorable factors). Are $X$ and $Y$ independent given $Z$? **Yes.** Knowing $Z = 1$, learning about study habits does not change the admission probability:

$$P(x, y \mid z) = \frac{P(x, y, z)}{P(z)} = \frac{P(y \mid z) P(z \mid x) P(x)}{P(z)} = \frac{P(x, z)}{P(z)} P(y \mid z) = P(x \mid z) P(y \mid z).$$

Observing $Z$ blocks the influence from $X$ to $Y$. (Marginally, $X$ and $Y$ are dependent.)

### 6.2 Common causes (forks)

$X \leftarrow Z \rightarrow Y$: rain $Z \leftarrow U_Z$ (meteorological conditions), traffic $X \leftarrow Z \vee \neg U_X$ (road design), umbrella $Y \leftarrow Z \wedge U_Y$. Are $X, Y$ independent? **No**: seeing an umbrella raises the probability of rain, which raises the probability of bad traffic; $\exists x, y$ with $P(x, y) \neq P(x)P(y)$. Information flows from $Y$ up to $Z$ and down to $X$. Given $Z$? **Yes**: once we know it is raining, umbrellas tell us nothing about traffic,

$$P(x, y \mid z) = \frac{P(x \mid z)P(y \mid z)P(z)}{P(z)} = P(x \mid z)P(y \mid z).$$

### 6.3 Common effects (colliders)

$X \rightarrow Z \leftarrow Y$: having a cold $X \leftarrow U_X$, holiday $Y \leftarrow U_Y$, going to class $Z \leftarrow \neg Y \wedge (\neg X \wedge U_Z)$ ($U_Z$: motivation). Are $X, Y$ independent? **Yes**:

$$P(x, y) = \sum_z P(x)P(y)P(z \mid x, y) = P(x)P(y) \sum_z P(z \mid x, y) = P(x)P(y).$$

Influence from $X$ reaches $Z$ but does not "go up" to $Y$. Given $Z$? **No!** If a student did not go to class ($Z = 0$) and it is not a holiday ($Y = 0$), a cold becomes more likely; $\exists x, y, z$ with $P(x, y \mid z) \neq P(x \mid z)P(y \mid z)$ ("explaining away"). This is the opposite of the previous two cases.

Given a descendant $W$ of the collider (signing the attendance sheet, $W \leftarrow Z \wedge U_W$)? **No, again.** Observing $W = 0$ makes absence ($Z = 0$) more likely, which again couples $X$ and $Y$. *Watch out for descendants of colliders.*

### 6.4 Triplet summary

| Triplet | Active (dependence flows) when | Inactive when |
|---|---|---|
| chain $X \rightarrow Z \rightarrow Y$ | $Z$ not observed | $Z$ observed |
| fork $X \leftarrow Z \rightarrow Y$ | $Z$ not observed | $Z$ observed |
| collider $X \rightarrow Z \leftarrow Y$ | $Z$ or a descendant of $Z$ observed | neither $Z$ nor its descendants observed |

### 6.5 d-separation

**Definition 6.1** (d-separation)**.** To decide whether $X \perp\!\!\!\perp Y \mid \mathbf{Z}$ (with $X, Y \notin \mathbf{Z}$):

1. look at every path between $X$ and $Y$;
2. a path is *active* given $\mathbf{Z}$ if every consecutive triplet on it is active;
3. if any path is active, $X$ and $Y$ are d-connected (not guaranteed independent); otherwise they are d-separated, written $(X \perp_d Y \mid \mathbf{Z})_{\mathcal{G}}$.

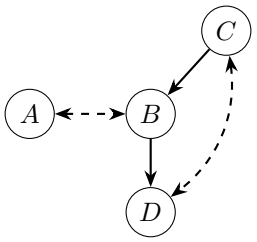
Trivially, if $X = Y$ or $X$ and $Y$ are adjacent, they are dependent.

Soundness: if $(X \perp_d Y \mid \mathbf{Z})_{\mathcal{G}}$ then $X \perp\!\!\!\perp Y \mid \mathbf{Z}$ in every distribution induced by an SCM compatible with $\mathcal{G}$ (the global Markov property, equivalent to the local one for DAGs).

**Example 6.2** (Sprinkler network)**.** Season → Sprinkler, Season → Rain, Sprinkler → Wet, Rain → Wet, Wet → Slippery.

1. Wet $\perp\!\!\!\perp$ Sprinkler? **No**, they are adjacent.
2. Wet $\perp\!\!\!\perp$ Season | Sprinkler? **No**: Season $\to$ Rain $\to$ Wet is open.
3. Rain $\perp\!\!\!\perp$ Slippery | Wet? **Yes**: the only path is the chain through Wet.
4. Season $\perp\!\!\!\perp$ Wet | Sprinkler, Rain? **Yes**: both directed paths are blocked at observed non-colliders.
5. Sprinkler $\perp\!\!\!\perp$ Rain | Season, Wet? **No**: Wet is an observed collider on Sprinkler $\to$ Wet $\leftarrow$ Rain.

**Example 6.3** (Food for thought)**.** Graph: $A \leftrightarrow B$, $C \to B$, $B \to D$, $C \leftrightarrow D$.




- $A \perp\!\!\!\perp D$? **No**: on $A \leftrightarrow B \to D$, $B$ is an unobserved non-collider.
- $A \perp\!\!\!\perp C$? **Yes**: $A \leftrightarrow B \leftarrow C$ has a collider at $B$; $A \leftrightarrow B \to D \leftrightarrow C$ has a collider at $D$.
- $A \perp\!\!\!\perp C \mid D$? **No**: $B$ is an ancestor of the observed $D$, so the collider at $B$ is opened.
- $D \perp\!\!\!\perp C \mid B$? **No**: $D \leftrightarrow C$ are adjacent (confounded).

# 7 Implementation: graph algorithms in Python

**Purpose.** Turn the definitions into code: topological ordering and d-separation.

## 7.1 Topological order

Idea: a DAG has a source node (no parents); removing a node and its edges leaves a DAG. Find a source, then recursively order the rest.

```
def topological_order(G):
    for V in G.Vs:                  # for every node
        if not G.Pa(V):             # if there is no parent for V
            return [V] + topological_order(G - V)
    return []
```

This naive version is quadratic and silently returns a partial order on cyclic graphs. Tracking in-degrees (Kahn's algorithm) is efficient and detects cycles:

```
def topological_order(G):
    order = deque()
    in_degrees = {V: len(G.Pa(V)) for V in G.Vs}
    while in_degrees:
        for v, v_deg in in_degrees.items():
            if v_deg == 0:
                break
        else:
            raise ValueError('cyclic')
        order.append(v)
```

```
        del in_degrees[v]
        for c in G.Ch(v):
            in_degrees[c] -= 1
    return order
```

The `for`–`else` clause runs only if no zero-in-degree node was found, which happens iff the remaining graph contains a cycle.

### 7.2 d-separation via reachability (Markovian graphs)

$X$ and $Y$ are d-connected given $\mathbf{Z}$ iff there is a path along which every triplet $A - B - C$ satisfies:

$$\begin{aligned} A \to B \to C \quad & \text{only if } B \notin \mathbf{Z}, \\ A \leftarrow B \leftarrow C \quad & \text{only if } B \notin \mathbf{Z}, \\ A \leftarrow B \to C \quad & \text{only if } B \notin \mathbf{Z}, \\ A \to B \leftarrow C \quad & \text{only if } B \in An(\mathbf{Z}). \end{aligned}$$

So we walk from $B$ to $C$ only if the condition holds, using BFS/DFS over states (direction of arrival, $B$): arrived along an edge pointing *into* $B$ ($\to B$) or *out of* $B$ ($\leftarrow B$).

```
def d_separation(G, X, Y, Zs):
    q = visited_deque({('->', X), ('<-', X)})
    while q:
        direction, B = q.pop()
        if B == Y:
            return False
        if direction == '->':          # arrived with arrowhead into B
            if B in G.An(Zs):          # collider: open iff B in An(Z)
                q.extend({('<-', C) for C in G.Pa(B)})
            if B not in Zs:            # chain continues
                q.extend({('->', C) for C in G.Ch(B)})
        else:                          # arrived via tail of B (B -> previous)
            if B not in Zs:
                q.extend({('->', C) for C in G.Ch(B)})
                q.extend({('<-', C) for C in G.Pa(B)})
    return True
```

`visited_deque` is a deque that ignores states already visited, so each (direction, node) pair is processed once and the algorithm is linear in the size of the graph. The starting states allow leaving $X$ in either direction.

## 8 Source coverage

The source deck has 130 physical pages (60 numbered slides with build-ups).

| Topic | Physical pages |
|---|---|
| Notation, probability and algebra reviews | 1–13 |
| Prima facie example, paradigms, modeling reality | 14–35 |
| SCM definition, Property 1 (induced distribution) | 36–48 |
| Property 2 (causal diagram), terminology | 49–60 |
| Property 3, first layer, topological order, factorization | 61–74 |
| Conditional independence, local Markov, Markovian factorization | 75–89 |
| Chains, forks, colliders, triplets, d-separation, exercises | 90–111 |
| Python implementation, references | 112–130 |